

3.4 Doubly Linked Lists

In a singly linked list, each node maintains a reference to the node that is immediately after it. We have demonstrated the usefulness of such a representation when managing a sequence of elements. However, there are limitations that stem from the asymmetry of a singly linked list. In Section 3.2, we demonstrated that we can efficiently insert a node at either end of a singly linked list, and can delete a node at the head of a list, but we are unable to efficiently delete a node at the tail of the list. More generally, we cannot efficiently delete an arbitrary node from an interior position of the list if only given a reference to that node, because we cannot determine the node that immediately *precedes* the node to be deleted (yet, that node needs to have its next reference updated).

To provide greater symmetry, we define a linked list in which each node keeps an explicit reference to the node before it and a reference to the node after it. Such a structure is known as a **doubly linked list**. These lists allow a greater variety of $O(1)$ -time update operations, including insertions and deletions at arbitrary positions within the list. We continue to use the term “next” for the reference to the node that follows another, and we introduce the term “prev” for the reference to the node that precedes it.

Header and Trailer Sentinels

In order to avoid some special cases when operating near the boundaries of a doubly linked list, it helps to add special nodes at both ends of the list: a **header** node at the beginning of the list, and a **trailer** node at the end of the list. These “dummy” nodes are known as **sentinels** (or guards), and they do not store elements of the primary sequence. A doubly linked list with such sentinels is shown in Figure 3.19.

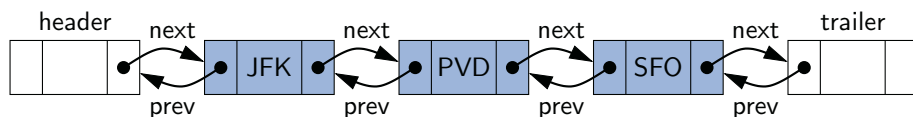


Figure 3.19: A doubly linked list representing the sequence { JFK, PVD, SFO }, using sentinels header and trailer to demarcate the ends of the list.

When using sentinel nodes, an empty list is initialized so that the next field of the header points to the trailer, and the prev field of the trailer points to the header; the remaining fields of the sentinels are irrelevant (presumably null, in Java). For a nonempty list, the header’s next will refer to a node containing the first real element of a sequence, just as the trailer’s prev references the node containing the last element of a sequence.

Advantage of Using Sentinels

Although we could implement a doubly linked list without sentinel nodes (as we did with our singly linked list in Section 3.2), the slight extra memory devoted to the sentinels greatly simplifies the logic of our operations. Most notably, the header and trailer nodes never change—only the nodes between them change. Furthermore, we can treat all insertions in a unified manner, because a new node will always be placed between a pair of existing nodes. In similar fashion, every element that is to be deleted is guaranteed to be stored in a node that has neighbors on each side.

For contrast, we look at our `SinglyLinkedList` implementation from Section 3.2. Its `addLast` method required a conditional (lines 39–42 of Code Fragment 3.15) to manage the special case of inserting into an empty list. In the general case, the new node was linked after the existing tail. But when adding to an empty list, there is no existing tail; instead it is necessary to reassign `head` to reference the new node. The use of a sentinel node in that implementation would eliminate the special case, as there would always be an existing node (possibly the header) before a new node.

Inserting and Deleting with a Doubly Linked List

Every insertion into our doubly linked list representation will take place between a pair of existing nodes, as diagrammed in Figure 3.20. For example, when a new element is inserted at the front of the sequence, we will simply add the new node *between* the header and the node that is currently after the header. (See Figure 3.21.)

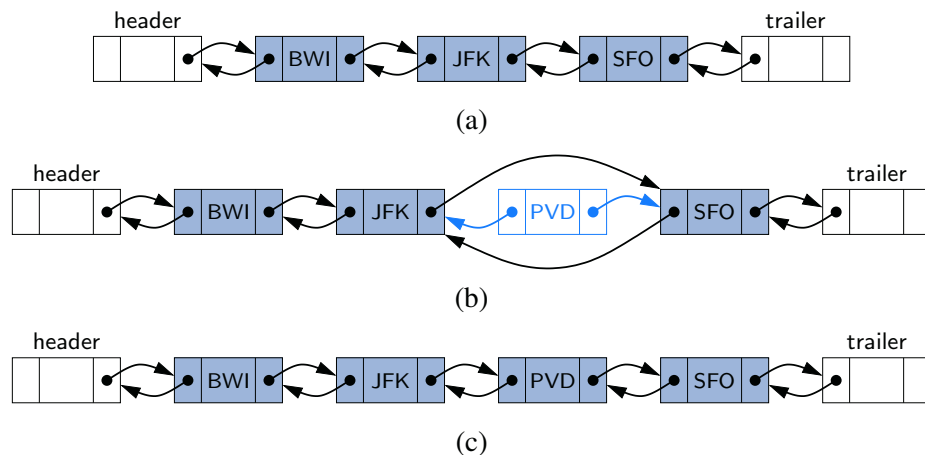


Figure 3.20: Adding an element to a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

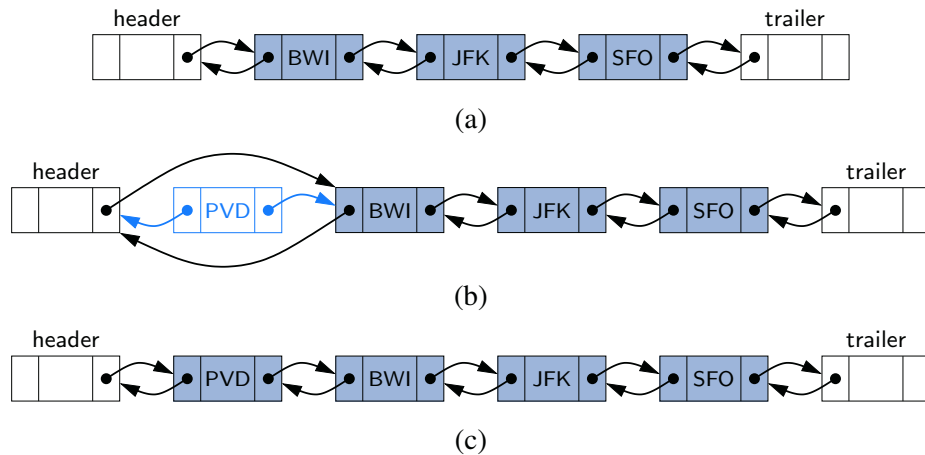


Figure 3.21: Adding an element to the front of a sequence represented by a doubly linked list with header and trailer sentinels: (a) before the operation; (b) after creating the new node; (c) after linking the neighbors to the new node.

The deletion of a node, portrayed in Figure 3.22, proceeds in the opposite fashion of an insertion. The two neighbors of the node to be deleted are linked directly to each other, thereby bypassing the original node. As a result, that node will no longer be considered part of the list and it can be reclaimed by the system. Because of our use of sentinels, the same implementation can be used when deleting the first or the last element of a sequence, because even such an element will be stored at a node that lies between two others.

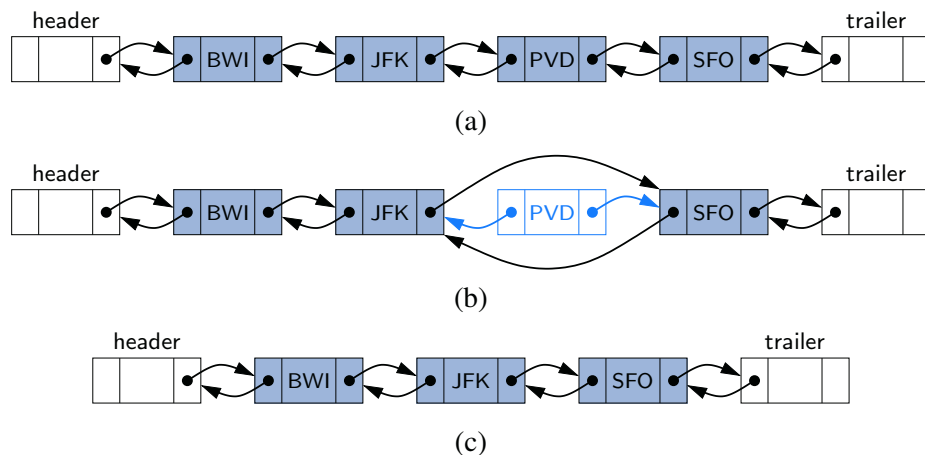


Figure 3.22: Removing the element PVD from a doubly linked list: (a) before the removal; (b) after linking out the old node; (c) after the removal (and garbage collection).

3.4.1 Implementing a Doubly Linked List Class

In this section, we present a complete implementation of a `DoublyLinkedList` class, supporting the following public methods:

`size()`: Returns the number of elements in the list.

`isEmpty()`: Returns **true** if the list is empty, and **false** otherwise.

`first()`: Returns (but does not remove) the first element in the list.

`last()`: Returns (but does not remove) the last element in the list.

`addFirst(e)`: Adds a new element to the front of the list.

`addLast(e)`: Adds a new element to the end of the list.

`removeFirst()`: Removes and returns the first element of the list.

`removeLast()`: Removes and returns the last element of the list.

If `first()`, `last()`, `removeFirst()`, or `removeLast()` are called on a list that is empty, we will return a **null** reference and leave the list unchanged.

Although we have seen that it is possible to add or remove an element at an internal position of a doubly linked list, doing so requires knowledge of one or more nodes, to identify the position at which the operation should occur. In this chapter, we prefer to maintain encapsulation, with a private, nested `Node` class. In Chapter 7, we will revisit the use of doubly linked lists, offering a more advanced interface that supports internal insertions and deletions while maintaining encapsulation.

Code Fragments 3.17 and 3.18 present the `DoublyLinkedList` class implementation. As we did with our `SinglyLinkedList` class, we use the generics framework to accept any type of element. The nested `Node` class for the doubly linked list is similar to that of the singly linked list, except with support for an additional `prev` reference to the preceding node.

Our use of sentinel nodes, header and trailer, impacts the implementation in several ways. We create and link the sentinels when constructing an empty list (lines 25–29). We also keep in mind that the first element of a nonempty list is stored in the node just *after* the header (not in the header itself), and similarly that the last element is stored in the node just *before* the trailer.

The sentinels greatly ease our implementation of the various update methods. We will provide a private method, `addBetween`, to handle the general case of an insertion, and then we will rely on that utility as a straightforward method to implement both `addFirst` and `addLast`. In similar fashion, we will define a private `remove` method that can be used to easily implement both `removeFirst` and `removeLast`.

```

1  /** A basic doubly linked list implementation. */
2  public class DoublyLinkedList<E> {
3      //----- nested Node class -----
4      private static class Node<E> {
5          private E element;           // reference to the element stored at this node
6          private Node<E> prev;        // reference to the previous node in the list
7          private Node<E> next;        // reference to the subsequent node in the list
8          public Node(E e, Node<E> p, Node<E> n) {
9              element = e;
10             prev = p;
11             next = n;
12         }
13         public E getElement() { return element; }
14         public Node<E> getPrev() { return prev; }
15         public Node<E> getNext() { return next; }
16         public void setPrev(Node<E> p) { prev = p; }
17         public void setNext(Node<E> n) { next = n; }
18     } //----- end of nested Node class -----
19
20     // instance variables of the DoublyLinkedList
21     private Node<E> header;           // header sentinel
22     private Node<E> trailer;          // trailer sentinel
23     private int size = 0;              // number of elements in the list
24     /** Constructs a new empty list. */
25     public DoublyLinkedList() {
26         header = new Node<>(null, null, null); // create header
27         trailer = new Node<>(null, header, null); // trailer is preceded by header
28         header.setNext(trailer); // header is followed by trailer
29     }
30     /** Returns the number of elements in the linked list. */
31     public int size() { return size; }
32     /** Tests whether the linked list is empty. */
33     public boolean isEmpty() { return size == 0; }
34     /** Returns (but does not remove) the first element of the list. */
35     public E first() {
36         if (isEmpty()) return null;
37         return header.getNext().getElement(); // first element is beyond header
38     }
39     /** Returns (but does not remove) the last element of the list. */
40     public E last() {
41         if (isEmpty()) return null;
42         return trailer.getPrev().getElement(); // last element is before trailer
43     }

```

Code Fragment 3.17: Implementation of the DoublyLinkedList class. (Continues in Code Fragment 3.18.)

```

44 // public update methods
45 /** Adds element e to the front of the list. */
46 public void addFirst(E e) {
47     addBetween(e, header, header.getNext()); // place just after the header
48 }
49 /** Adds element e to the end of the list. */
50 public void addLast(E e) {
51     addBetween(e, trailer.getPrev(), trailer); // place just before the trailer
52 }
53 /** Removes and returns the first element of the list. */
54 public E removeFirst() {
55     if (isEmpty()) return null; // nothing to remove
56     return remove(header.getNext()); // first element is beyond header
57 }
58 /** Removes and returns the last element of the list. */
59 public E removeLast() {
60     if (isEmpty()) return null; // nothing to remove
61     return remove(trailer.getPrev()); // last element is before trailer
62 }
63
64 // private update methods
65 /** Adds element e to the linked list in between the given nodes. */
66 private void addBetween(E e, Node<E> predecessor, Node<E> successor) {
67     // create and link a new node
68     Node<E> newest = new Node<>(e, predecessor, successor);
69     predecessor.setNext(newest);
70     successor.setPrev(newest);
71     size++;
72 }
73 /** Removes the given node from the list and returns its element. */
74 private E remove(Node<E> node) {
75     Node<E> predecessor = node.getPrev();
76     Node<E> successor = node.getNext();
77     predecessor.setNext(successor);
78     successor.setPrev(predecessor);
79     size--;
80     return node.getElement();
81 }
82 } //----- end of DoublyLinkedList class -----

```

Code Fragment 3.18: Implementation of the public and private update methods for the `DoublyLinkedList` class. (Continued from Code Fragment 3.17.)